Lab-1

Title- Apply basic feature engineering techniques such as handling missing values, encoding categorical variables, and scaling numerical attributes on the suitable open access dataset to prepare a clean and model ready dataset for machine learning analysis.

Objectives

1. To understand the importance of feature engineering in machine learning.
2. To identify and handle missing values in a dataset.
3. To encode categorical variables into numerical form.
4. To scale numerical attributes using standard feature scaling techniques.
5. To prepare a clean and machine learning-ready dataset for further analysis.

Problem Statement

Apply basic feature engineering techniques such as handling missing values, encoding categorical variables, and scaling numerical attributes on a suitable open-access dataset to prepare a clean and model-ready dataset for machine learning analysis.

Software Requirements

- Python 3.x
- Jupyter Notebook / Google Colab / VS Code
- Pandas
- NumPy
- Scikit-learn
- Matplotlib
- Seaborn

Hardware Requirements

- Processor: Intel Core i3 or higher
- RAM: 4 GB minimum (8 GB recommended)
- Hard Disk: 10 GB free space
- Operating System: Windows/Linux/macOS

Theory

Feature Engineering and Data Preprocessing in Machine Learning

Machine learning models don't understand raw data the way humans do they understand numbers, patterns, and relationships expressed in a form the algorithm can compute on. Feature engineering is the bridge between messy, real-world information and the clean, structured numerical input a model actually consumes. It involves selecting, transforming, combining, and creating variables (features) so that the underlying signal in the data becomes easier for the model to learn.

1. Handling Missing Values

Real-world data is rarely complete. Sensors fail, survey respondents skip questions, systems log errors instead of values. Missing data can bias a model or cause outright failures in algorithms that can't handle NaNs.

Common strategies:

- **Deletion** — Removing rows or columns with missing values. Simple, but risky if the missingness isn't random (this can introduce bias) or if too much data is lost.
- **Imputation** — Filling in missing values using:
    - **Mean/median/mode imputation** — Simple and fast, works well for small amounts of missing data.
    - **Forward/backward fill** — Common in time-series data, where the last known value carries forward.
    - **Model-based imputation** — Using regression, k-nearest neighbors (KNN), or even other ML models to predict the missing value based on other features.
    - **Multiple Imputation (e.g., MICE)** — Statistically robust; generates several plausible values and averages results.
- **Indicator/flag variables** — Adding a binary column indicating whether a value was missing, which preserves the information that "missingness" itself might be meaningful (e.g., a missing income field might correlate with a certain demographic).

2. Standardizing Inconsistent Formats

Data collected from multiple sources (databases, APIs, manual entry, scraped web pages) often comes in inconsistent formats:

- Dates written as "2023-01-05," "01/05/2023," or "Jan 5, 2023"
- Text with inconsistent capitalization, whitespace, or typos ("USA" vs "U.S.A." vs "United States")
- Units mismatched (kilometers vs. miles, USD vs. EUR)
- Duplicate records with slightly different spellings

Preprocessing steps typically include:

- Parsing and standardizing date/time formats into a single consistent type
- Normalizing text (lowercasing, trimming whitespace, removing special characters)
- Converting units to a common standard
- Deduplication and fuzzy matching to merge near-identical records

3. Encoding Categorical Variables

Most ML algorithms (especially those based on linear algebra, like linear regression or neural networks) require numerical input. Categorical variables like "Color: Red/Blue/Green" or "City: Delhi/Mumbai/Chennai" must be converted into numeric form.

Common encoding techniques:

- **One-Hot Encoding** — Creates a binary column for each category. Good for nominal (unordered) categories, but can cause high dimensionality if there are many unique categories.
- **Label Encoding** — Assigns an integer to each category. Works well for ordinal data (e.g., "Low, Medium, High") but can mislead models into assuming a false numeric relationship for nominal data.
- **Ordinal Encoding** — Similar to label encoding, but explicitly respects a meaningful order.
- **Target Encoding / Mean Encoding** — Replaces categories with the mean of the target variable for that category. Powerful but prone to overfitting/data leakage if not done carefully (e.g., using cross-validation folds).
- **Frequency Encoding** — Replaces categories with how often they occur in the dataset.
- **Embeddings** — Used in deep learning, where categories (especially high-cardinality ones like product IDs or words) are mapped into dense, learned vector representations.

4. Scaling Features with Different Ranges

Features often exist on very different scales — for example, "Age" might range from 0–100, while "Income" might range from 0–1,000,000. Many algorithms (like KNN, SVM, gradient descent-based models, and neural networks) are sensitive to these scale differences because they use distance metrics or gradient magnitudes that get dominated by larger-scale features.

Common scaling techniques:

- **Min-Max Normalization** — Rescales values into a fixed range, usually [0,1]. Sensitive to outliers.
- **Standardization (Z-score scaling)** — Transforms data to have a mean of 0 and standard deviation of 1. Preferred when data is roughly normally distributed.
- **Robust Scaling** — Uses median and interquartile range instead of mean/standard deviation, making it resistant to outliers.
- **Log Transformation** — Useful for reducing skewness in data with long tails (e.g., income, population).
- **Unit Vector Scaling (Normalization)** — Scales feature vectors to have unit norm, often used in text or high-dimensional data.

Conclusion

Feature engineering significantly improves data quality and prepares datasets for machine learning algorithms. Handling missing values, converting categorical variables into numerical form, and scaling numerical features ensure better model accuracy, faster convergence, and more reliable predictions.

Viva Questions

1. What is feature engineering?
2. Why is handling missing data important?
3. What is the difference between mean, median, and mode imputation?
4. What is One-Hot Encoding?
5. What is Label Encoding?
6. Why is feature scaling required?
7. What is the difference between normalization and standardization?
8. Which machine learning algorithms require feature scaling?
9. What is StandardScaler?
10. What are the advantages of feature engineering?